# Title of Project Report

Group number: Student 1 Student 2 3

October 17, 2026

**Abstract**

An abstract is a short summary of a research paper or a report. Abstract is to help your reader understand quickly what your report is about. Abstract typically spends:

- importance of research and your problem (Introduction)
- what you did (Methods)
- what you have found (Results)
- discuss the implication and significance of the project (Discussion)

## 1 Introduction

Problem description, purpose of this report, and (if any) background of the data structures and the algorithms. Be concise.

## 2 Data Structure / Algorithm Specification

Description (pseudo-code preferred) of all the algorithms involved for solving the problem, including specifications of main data structures. Or, if you are to introduce a new data structure and its related operations, do it in this chapter.

If the algorithm you used is adopted from previous work, please cite that work, such as our textbook [**?**].

## 3 Testing Results

Table of test cases. Each test case usually consists of a brief description of the purpose of this case, the expected result, the actual behavior of your program, the possible cause of a bug if your program does not function as expected, and the current status ("pass", or "corrected", or "pending").

Example table in Table 1:

Table 1: Relevance

| | **Relevant** | **Irrelevant** |
|---|---|---|
| Retrieved | $R_R$ (true positive) | $I_R$ (false positive) |
| Not Retrieved | $R_N$ (false negative) | $I_N$ (true negative) |

## 4 Analysis and Comments

Analysis of the time and space complexities of the algorithms. Comments on comparing with other known data structures and algorithms. Further possible improvements.

## 5 Author list

Specify who did what to show that particular contributors deserve to have their names printed in the cover page of your report.

## Declaration

We hereby declare that all the work done in this project titled "XXX" is of our independent effort as a group.

## 6 Signatures

Each author must sign his/her name here.

Please keep in mind that these are the "minimum" requirements. Other requirements will be specified according to each project assignment.

# A Source Code (if required)

At least 30% of the lines must be commented. Otherwise the code will NOT be evaluated.

```
        static Position
        SingleRotateWithLeft( Position K2 )
        {
            Position K1;

            K1 = K2->Left;
            K2->Left = K1->Right;
            K1->Right = K2;

            K2->Height = Max( Height( K2->Left ), Height( K2->Right ) ) + 1;
            K1->Height = Max( Height( K1->Left ), K2->Height ) + 1;

            return K1;  /* New root */
        }
```

Pseudocode of Dijkstra's algorithm (Heap-Based)

```
for every vertex v in V do
    d_v ← ∞; p_v ← null
    Insert(Q, v, d_v)  /* initialize vertex priority in the priority queue */
d_s ← 0; Decrease(Q, s, d_s)  /* update priority of s with d_s */
V_T ← ∅
for i ← 0 to |V| − 1 do
    u* ← DeleteMin(Q)  /* delete the minimum priority element */
    V_T ← V_t ⋃ {u*}
    for every vertex u in V − V_T  that is adjacent to u* do
        if d_u* + w(u*, u) < d_u
            d_u ← d_u* + w(u*, u);  p_u ← u*
            Decrease(Q, u, d_u)
```